

1. University Course Registration System

A university database contains the following tables:

- **Student** (StudentID, Name, DepartmentID)
- **Department** (DepartmentID, DepartmentName)
- **Course** (CourseID, CourseName, FacultyID)
- **Faculty** (FacultyID, FacultyName)
- **Enrollment** (StudentID, CourseID, Semester)

A student can enroll in multiple courses, and each course can have multiple students.

Viva Questions

Q1. Why is the Enrolment table necessary instead of storing CourseID directly in the Student table?

The **Enrollment** table is required because the relationship between **Student** and **Course** is **many-to-many**.

- A **student** can enroll in **multiple courses**.
- A **course** can have **multiple students**.

If CourseID were stored directly in the Student table:

- A student could store only one course (or multiple columns like Course1, Course2, which is not normalized).
- It would lead to data redundancy and make updates difficult.

The **Enrollment(StudentID, CourseID, Semester)** table:

- Resolves the many-to-many relationship.
- Eliminates redundancy.
- Allows a student to enroll in any number of courses.
- Stores additional enrollment details such as the semester.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

Add a **Grade** attribute to the **Enrollment** table because the grade belongs to a specific student in a specific course.

Modified Enrollment Table:

Enrollment(StudentID, CourseID, Semester, Grade)

Example:

StudentID	CourseID	Semester	Grade
S101	C201	Fall 2026	A
S101	C202	Fall 2026	B+
S102	C201	Fall 2026	A-

2. Hospital Management System

A hospital database contains:

- **Patient** (PatientID, Name, Age)
- **Doctor** (DoctorID, DoctorName, Specialization)
- **Appointment** (AppointmentID, PatientID, DoctorID, Date)
- **Prescription** (PrescriptionID, AppointmentID, Medicine)

Multiple patients can consult the same doctor.

Viva Questions

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

Answer:

Storing prescription details directly in the **Appointment** table can cause:

- **Data redundancy** if multiple medicines are prescribed.

- **Difficulty storing multiple medicines** for one appointment.
- **Violation of database normalization.**
- **Update and maintenance issues**, as modifying prescriptions becomes more complex.

Therefore, prescription details are stored in a separate **Prescription** table.

Q2. How would the schema change if a prescription contains multiple medicines?

Answer:

If one prescription contains multiple medicines, create a separate table to store each medicine.

Modified Schema:

Patient(PatientID, Name, Age)

Doctor(DoctorID, DoctorName, Specialization)

Appointment(AppointmentID, PatientID, DoctorID, Date)

Prescription(PrescriptionID, AppointmentID)

Prescription_Medicine(PrescriptionID, MedicineID, Dosage, Frequency)

Medicine(MedicineID, MedicineName)

PrescriptionID	MedicineID	Dosage	Frequency
PR101	M001	500 mg	Twice a day
PR101	M002	250 mg	Once a day

A single prescription may contain **multiple medicines**, so a separate **Prescription_Medicine** table represents the **one-to-many relationship** between prescriptions and medicines.

3. Online Shopping System

A database contains the following tables:

- **Customer** (CustomerID, Name)
- **Product** (ProductID, ProductName, Price)
- **Order** (OrderID, CustomerID, OrderDate)
- **OrderDetails** (OrderID, ProductID, Quantity)

Each order may contain multiple products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

Answer:

The **OrderDetails** table is required because the relationship between **Order** and **Product** is **many-to-many**.

- One order can contain multiple products.
- One product can appear in multiple orders.
- The OrderDetails table stores the **ProductID** and **Quantity** for each product in an order.
- It avoids data redundancy and maintains database normalization.

Q2. What would happen if ProductID were stored directly in the Order table?

Answer:

If **ProductID** were stored directly in the **Order** table:

- An order could store only one product.
- Multiple ProductID columns would be needed for multiple products, violating normalization.
- Data redundancy and update anomalies would occur.

- It would be difficult to store the quantity of each product.

Therefore, the **OrderDetails** table is necessary.

Existing Schema

Customer(CustomerID, Name)

Product(ProductID, ProductName, Price)

Order(OrderID, CustomerID, OrderDate)

OrderDetails(OrderID, ProductID, Quantity)

Example:

OrderID	ProductID	Quantity
---------	-----------	----------

O101	P201	2
------	------	---

O101	P305	1
------	------	---

O102	P201	3
------	------	---

The **OrderDetails** table acts as a bridge table that represents the **many-to-many relationship** between **Order** and **Product** while storing the quantity of each product ordered.